

Database Normalization Activity

Group 11

Big3 Construction Dataset

Members

Karabo Innocent Pamela, Second Normal Form

Rwema Gisa, First Normal Form

Sonia Wisdom Etuhoko, Boyce Codd Normal Form

Ange Umutoni, Third Normal Form

Irene Wanjiru Winnie, Fourth Normal Form and Documentation

We started with one raw CSV file containing construction project data, and the goal of this project was to take that file through five stages of normalization, first normal form, second normal form, third normal form, Boyce Codd normal form, and fourth normal form, cleaning up the structure at each stage until we ended up with a set of tables that properly represent the real entities in the data without unnecessary repetition. This document walks through what we found and what we did at each stage.

Looking at the raw data

Before touching any of the normal forms, we sat down and worked out what the raw file was actually describing. The data covers seven real entities. There are projects, which are the construction jobs themselves. There are clients, the companies or organizations that own each project. There are supervisors, who manage the projects. There are workers, who get assigned to projects and bring their own skills and certifications with them. There are suppliers, who provide materials and equipment to projects. And there are materials and equipment as their own separate ideas, since a project can use several different materials and several different pieces of equipment, each with its own cost.

The biggest problem with the raw file was that several columns held more than one value crammed into a single cell, separated by a vertical bar. WorkerSkills held more than one skill per worker, something like Carpentry and Framing in the same cell. WorkerCertifications did the same thing with certifications. SupplierPhones held more than one phone number for the same supplier. MaterialSupplied held a list of materials used on a project, and MaterialUnitCost held a matching list of costs for each of those materials. EquipmentUsed and EquipmentRentalCost worked the same way for equipment.

On top of that, the raw file repeated information constantly. Every time a project had more than one worker, the project's name, dates, site address, client information, and supplier information all got copied again on the new row, just because a different worker was being listed. The same project details, client details, and supplier details show up over and over depending on how many workers were on that job.

Looking at the relationships in the data, a project belongs to exactly one client, but a client can have many projects, so that is a many to one relationship. The same is true between a project and its supervisor. A project can have many workers and a worker can be on many projects, so that one is many to many. The same many to many pattern shows up between projects and suppliers, and between projects and equipment. Within a single worker, the relationship to skills is one to many, since one worker can have several skills, and the same goes for certifications.

That overview is what shaped every decision in the stages that follow.

First normal form

The first step was making sure every cell in the table held exactly one value. The seven columns listed above, WorkerSkills, WorkerCertifications, SupplierPhones, MaterialSupplied, MaterialUnitCost, EquipmentUsed, and EquipmentRentalCost, all needed to be split apart.

We did this in Python using pandas. For the simple multi valued columns like WorkerSkills, WorkerCertifications, and SupplierPhones, we split each cell on the vertical bar and then exploded the column, which turns one row with several values into several rows, each holding one value. For MaterialSupplied and MaterialUnitCost, and separately for EquipmentUsed and EquipmentRentalCost, the two columns needed to be split together rather than separately, since each material has to stay matched up with its own correct cost rather than getting mixed up with a different material's cost. We wrote a small function that splits both columns at once and pairs them back up correctly before exploding.

After this step, the original fifteen rows turned into two hundred and eighty rows, and every cell in the resulting table held a single atomic value. We did not create any new tables at this stage, everything still lived in one wide table, we just multiplied the rows so that each one represented a single combination of project, worker, skill, certification, supplier phone, material, and equipment. The row is uniquely identified by the full combination of ProjectID, WorkerName, Skill, Certification, SupplierPhone, MaterialSupplied, and EquipmentUsed together, since no two rows share the exact same combination of all of those values.

Second normal form

Once the data was atomic, the next problem was that a lot of columns only depended on part of that long composite key, not the whole thing. That is what second normal form is meant to catch.

Project details like the project name, type, start date, end date, and site address only depend on ProjectID, they have nothing to do with which worker or which skill is attached to that row. Client phone, email, and city only depend on the client, not on the project or worker. Supervisor phone only depends on the supervisor. Worker phone and hourly rate only depend on the worker, not on which project they happen to be working on. A worker's skill and certification are also properties of the worker alone, not of the project assignment. Supplier city and supplier phone numbers only depend on the supplier, not on which project is using that supplier. Material cost and equipment rental cost are a little different, since those genuinely depend on the full pairing of project and material, or project and equipment, because the same item can cost different amounts on different jobs.

Based on that, we split the single wide table into twelve separate tables. Projects, clients, supervisors, and workers each became their own table holding only the details that depend on that one entity. Project workers became a table that just links a project

to a worker, with nothing else in it. Worker skills and worker certifications became their own two tables, each linking a worker to one skill or one certification. Suppliers became its own table, supplier phones became a separate table since a supplier can have more than one phone number, and project suppliers links a project to a supplier. Project materials and project equipment kept the cost columns, since that cost depends on the full project and item pairing rather than on just one side of it.

With that split, every non key column in every table depends on the whole key of that table, not just part of it, which is exactly what second normal form requires.

Third normal form

The next thing to check for is transitive dependencies, which is when one non key column determines another non key column inside the same table.

We found six of these. In the clients table, ClientName was determining ClientPhone, ClientEmail, and ClientCity, so we gave clients their own ClientID and pulled those contact details out into a proper clients table keyed on ClientID. The same pattern showed up with SupervisorName determining SupervisorPhone, with WorkerName determining WorkerPhone and WorkerHourlyRate, with SupplierName determining SupplierCity, with MaterialName determining MaterialUnitCost, and with EquipmentName determining EquipmentRentalCost. In each case we gave that entity its own ID, moved the dependent details into a table keyed on that ID, and replaced the name with the ID wherever it had been used as a reference.

That gave us the same fourteen tables we use going forward, projects, clients, supervisors, workers, suppliers, materials, and equipment as entity tables, and project workers, project suppliers, project materials, project equipment, worker skills, worker certifications, and supplier phones as the tables that capture the relationships between them. Every column in every table now depends only on the primary key of that table and nothing else, so every one of the earlier transitive dependencies is gone, and all of the original relationships are still fully represented through the foreign keys connecting these tables to each other.

Boyce Codd normal form

Boyce Codd normal form is stricter than third normal form. The rule is that for every functional dependency in a table, the column on the left side of that dependency has to be a candidate key, not just any column that happens to determine something.

We went through every one of the fourteen tables and checked this. Most of them passed without any changes. Projects, clients, supervisors, project workers, worker skills, worker certifications, and project suppliers were all fine, because in each of those tables, the only determinant is the actual key.

The two tables that did not pass were materials and equipment. In the materials table, the same material name does not always carry the same cost. Concrete costs 120 on one project and 110 on another, and 130 on a third. Storing one fixed cost per material

name forces a single value and throws away all the other real costs, so MaterialName cannot be treated as something that determines a single unit cost on its own. The same issue showed up with equipment, where a crane rents for a different price depending on which project is using it.

To fix this, we removed the cost column from both materials and equipment, leaving those tables with nothing but the ID and the name. The cost moved into project materials and project equipment instead, where it depends on the full project and item key rather than on the item alone. After that change, every determinant in every table is a candidate key, project specific costs are preserved correctly instead of being collapsed into one number, and the database satisfies Boyce Codd normal form.

Fourth normal form

Fourth normal form asks whether any table is still combining two unrelated many to many facts about the same thing into one place.

We checked every table coming out of the Boyce Codd stage for this. The biggest example in this dataset is at the project level. A project's list of workers has nothing to do with its list of suppliers, or its list of materials, or its list of equipment, those are four separate facts about the same project. The second example is at the worker level, since a worker's skills have nothing to do with that same worker's certifications.

By the time the Boyce Codd tables were finished, both of those had already been split apart properly, project workers, project suppliers, project materials, and project equipment are four separate tables, and worker skills and worker certifications are two separate tables as well. Going table by table, none of the fourteen tables hold two independent lists at once, each one holds exactly one such list tied to its own key, with cost columns where that applies depending on the full key rather than acting as a second hidden list. That means the fourth normal form tables are the same fourteen tables produced at the Boyce Codd stage, carried forward unchanged, because checking them properly did not turn up anything left to fix. Joining project workers, project suppliers, project materials, and project equipment back together on ProjectID reconstructs exactly the original set of project associations, with nothing extra and nothing missing, which is the actual test for whether a decomposition like this was done correctly.

Where we ended up

Across the five stages, the raw file went from one wide table with repeated information and multiple values crammed into single cells, to fourteen clean tables, each representing either one real entity or one real relationship between entities, with every dependency properly accounted for at each step.